

# Lab Program

## EX: 1. 8-Puzzle Problem

### Aim

To write a Python program to solve the 8-Puzzle problem using the A\* Search Algorithm.

### Objective

- To place **8 queens** on an **8 × 8 chessboard**.
- Ensure that **no two queens attack each other**.
- Use the **Backtracking Algorithm** to find a valid arrangement.

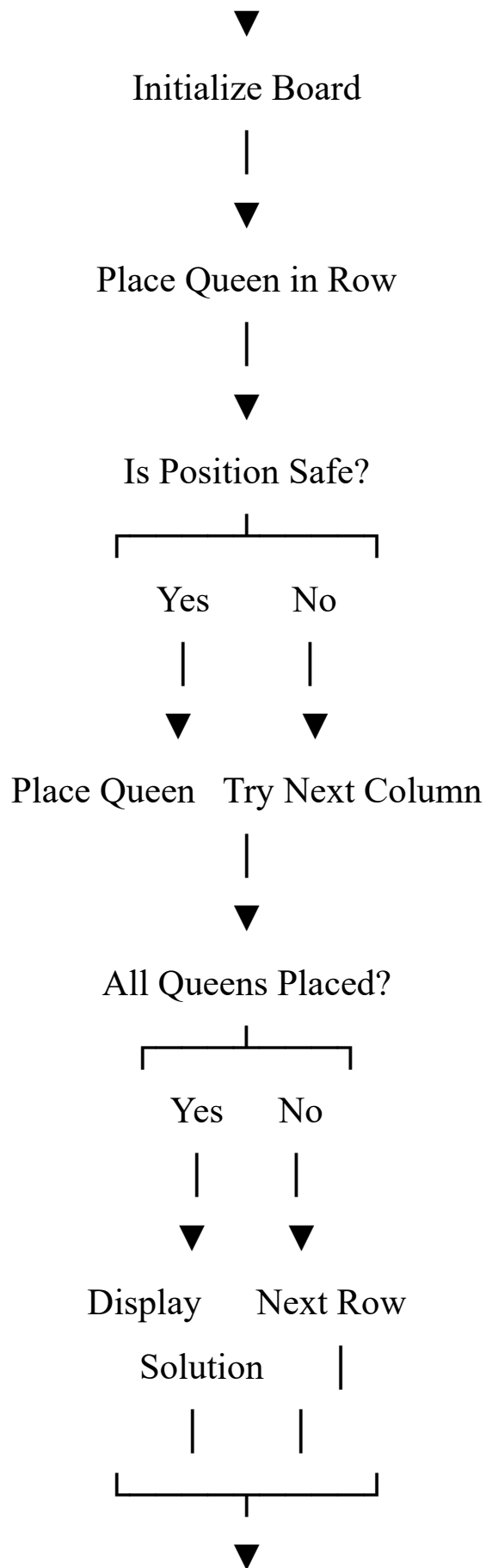
### Algorithm

1. Define the initial and goal states.
2. Find the blank space (0).
3. Move the blank tile in possible directions.
4. Calculate the heuristic value.
5. Repeat until the goal state is reached.
6. Display each step.

### Flowchart

Start

|



End

## **Pseudo Code**

START

Initialize start and goal state

Find possible moves

Repeat

    Select best move

    Display current state

Until goal state is reached

STOP

## **Python program**

```
from queue import PriorityQueue
```

```
start = ((1,2,3),(4,0,6),(7,5,8))
```

```
goal = ((1,2,3),(4,5,6),(7,8,0))
```

```
def moves(s):
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            if s[i][j] == 0:
```

```
                x, y = i, j
```

```

for dx, dy in [(-1,0),(1,0),(0,-1),(0,1)]:
    nx, ny = x+dx, y+dy
    if 0 <= nx < 3 and 0 <= ny < 3:
        t = [list(r) for r in s]
        t[x][y], t[nx][ny] = t[nx][ny], t[x][y]
        yield tuple(map(tuple, t))

pq = PriorityQueue()
pq.put((0, start))
visited = set()
print("Steps to Solve the 8-Puzzle:\n")

```

```

while not pq.empty():

```

```

    _, state = pq.get()

```

```

    if state in visited:

```

```

        continue

```

```

    visited.add(state)

```

```

    for row in state:

```

```

        print(row)

```

```

    print()

```

```

    if state == goal:

```

```

        break

```

```

    for nxt in moves(state):

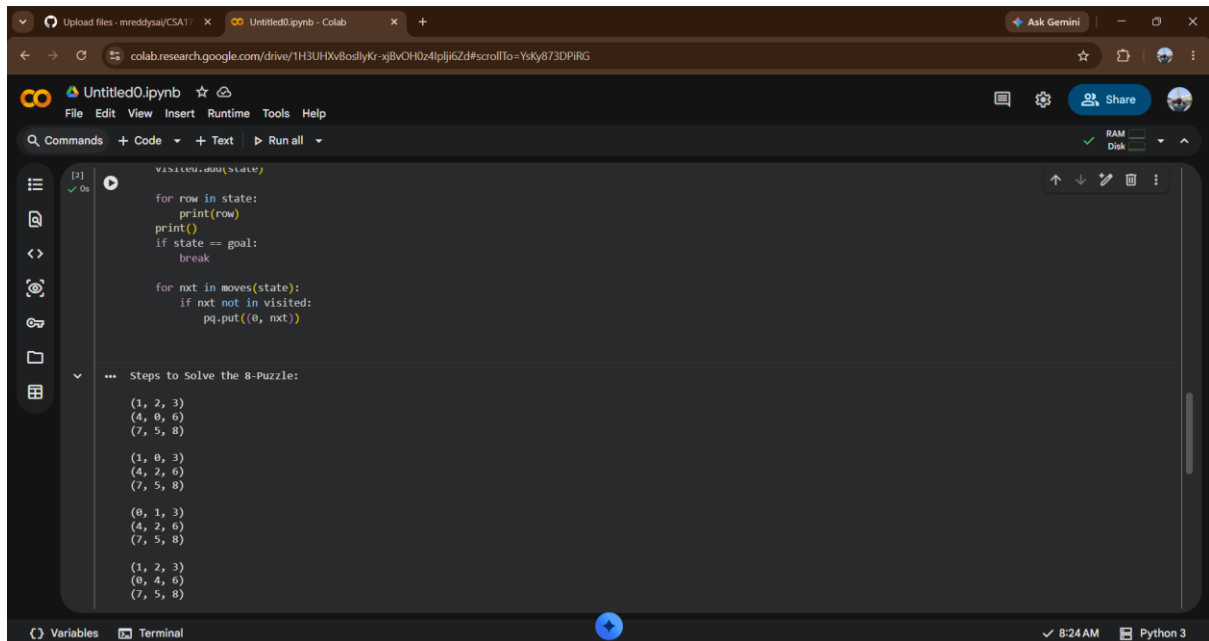
```

if nxt not in visited:

    pq.put((0, nxt))

**output :**

-



```
visited.add(state)
for row in state:
    print(row)
print()
if state == goal:
    break

for nxt in moves(state):
    if nxt not in visited:
        pq.put((0, nxt))
```

Steps to Solve the 8-Puzzle:

```
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 0, 3)
(4, 2, 6)
(7, 5, 8)

(0, 1, 3)
(4, 2, 6)
(7, 5, 8)

(1, 2, 3)
(0, 4, 6)
(7, 5, 8)
```

Result:the program was successfully implemented

## EX: 2. 8-Queen Problem

### Aim

To write a Python program to solve the 8-Queen Problem using the Backtracking Algorithm.

### Algorithm

1. Start from the first column.
2. Check whether the queen can be placed safely.

3. If safe, place the queen.
4. Move to the next column.
5. If no position is safe, backtrack.
6. Print the final board.

## **Pseudo Code:**

START

Create an empty board

Place queen column by column

If safe

    Place queen

Else

    Backtrack

Print solution

STOP

## **Python Program:**

N = 8

board = [["." for \_ in range(N)] for \_ in range(N)]

def safe(row,col):

    for i in range(col):

        if board[row][i]=="Q":

```

        return False
    i,j=row,col
    while i>=0 and j>=0:
        if board[i][j]=="Q":
            return False
        i-=1
        j-=1
    i,j=row,col
    while i<N and j>=0:
        if board[i][j]=="Q":
            return False
        i+=1
        j-=1
    return True
def solve(col):
    if col==N:
        return True

    for i in range(N):
        if safe(i,col):
            board[i][col]="Q"

            if solve(col+1):
                return True

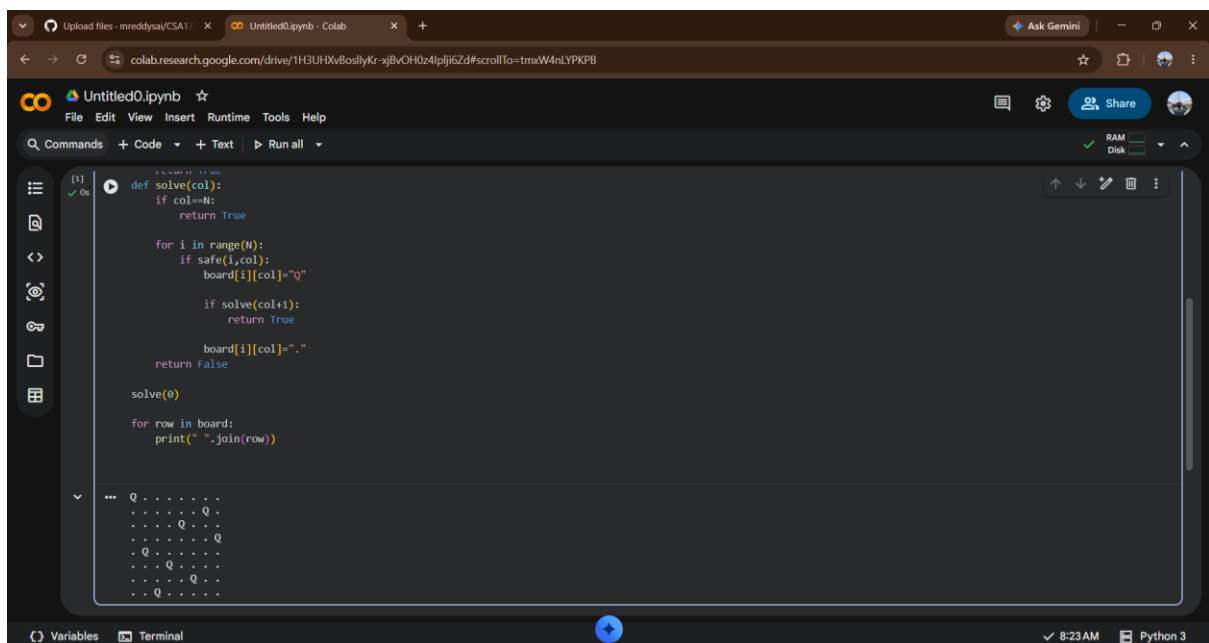
```

```
        board[i][col]="."
    return False
```

```
solve(0)
```

```
for row in board:
    print(" ".join(row))
```

**Output :**



```
def solve(col):
    if col==N:
        return True

    for i in range(N):
        if safe(i,col):
            board[i][col]="Q"

            if solve(col+1):
                return True

            board[i][col]="."
    return False

solve(0)

for row in board:
    print(" ".join(row))
```

... Q . . . . .  
. . . . . Q .  
. . . . . Q .  
. . . . . Q .  
. Q . . . . .  
. . . . . Q .  
. . . . . Q .  
. Q . . . . .

Result:the program was successfully implemented